

Resistive RAM With Multiple Bits Per Cell: Array-Level Demonstration of 3 Bits Per Cell

Binh Q. Le¹, Alessandro Grossi, Elisa Vianello, *Member, IEEE*, Tony Wu¹, Giusy Lama, Edith Beigné², H.-S. Philip Wong¹, and Subhasish Mitra¹

Abstract—We demonstrate, for the first time, resistive RAM (RRAM) arrays where each cell can store 3 bits. Such full array-level demonstration is possible through special techniques (e.g., that exploit RRAM-specific characteristics of variations in cell resistances), presented in this paper, which efficiently allocate resistance range corresponding to each bit combination (required for proper write operation) while maintaining appropriate sensing margin (required for proper read operation). Our techniques are not restricted to 3 bits-per-cell only (and, in this paper, we demonstrate 2 bits-per-cell at the array level as well). Our measured results are based on multiple 4 Kbit arrays of 1T1R HfO_x-based RRAM integrated in the back end of the line of 130-nm silicon CMOS technology. We achieve 3 bits-per-cell (2 bits-per-cell) RRAM with 11 (3) programming iterations on average.

Index Terms—Bitline voltage adaptation, HfO₂, machine learning, performance, resistance distribution allocation, resistive RAM (RRAM), sensing margin.

I. INTRODUCTION

STORING multiple bits inside a single-memory cell, referred to as multiple bits-per-cell memory, can be an effective way of improving storage density of resistive RAM (RRAM) [2], [3], the memory type used in this paper. Fig. 1 shows the technology and the integration flow for our 4 Kbit 1T1R HfO_x-based RRAM arrays, Fig. 2 shows the measured switching characteristics, and Fig. 3 shows the overall array architecture.

For illustration purposes, we discuss 2 bits-per-cell RRAM starting with Fig. 4 (later in this paper, we show the results for 3 bits-per-cell RRAM). To realize RRAM with multiple bits per cell, we must assign a resistance range (within the resistance window) to each bit (binary) combination (Fig. 4).

For array-level demonstrations, cell-to-cell variations must be taken into account; i.e., the resistance range corresponding

Manuscript received September 28, 2018; revised November 1, 2018; accepted November 2, 2018. Date of publication November 26, 2018; date of current version December 24, 2018. This work was supported in part by DARPA and in part by the Stanford SystemX Alliance. The review of this paper was arranged by Editor P. Du. (Corresponding author: Binh Q. Le.)

B. Q. Le, T. Wu, H.-S. P. Wong, and S. Mitra are with the Department of Electrical Engineering, Stanford University, Stanford, CA 94305 USA, also with the Department of Computer Science, Stanford University, Stanford, CA 94305 USA, and also with the Stanford SystemX Alliance, Stanford University, Stanford, CA 94305 USA (e-mail: binh@stanford.edu).

A. Grossi, E. Vianello, G. Lama, and E. Beigne are with CEA-Leti, 8054 Grenoble, France.

Digital Object Identifier 10.1109/TED.2018.2879788

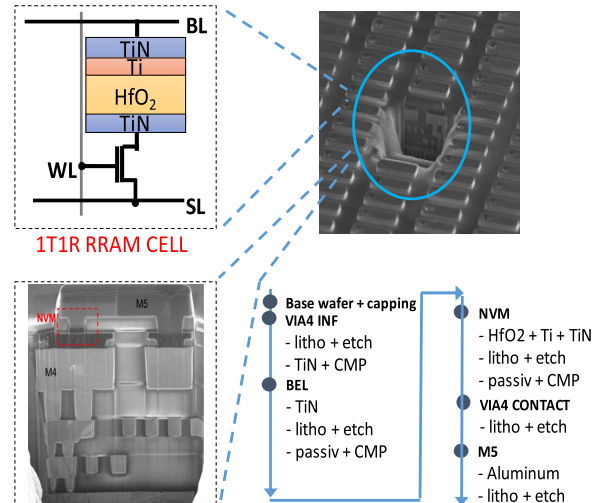


Fig. 1. RRAM integration flow and SEM cross section of the array used in this paper.

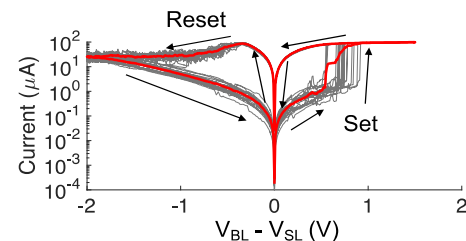


Fig. 2. Measured RRAM cell switching characteristics.

to a bit combination must be aware of the cell resistance distribution. Due to various variations, dominated by cell resistance variation in RRAM technology [7] (discussed in Section II), all cells cannot be programmed to a single-resistance value, even if the same programming pulse is applied to the cells during write (programming) operation [Fig. 5(a)]. In addition, there must also be resistance gaps between resistance ranges so that a read operation [Fig. 5(b)] can determine which resistance range a given RRAM cell belongs to. Thus, to achieve reliable multiple bits-per-cell RRAM at the array level (taking into account process variations), we rely on correct allocation of resistance ranges, correct allocation of resistance gaps, and an iterative programming scheme [Fig. 5(a)].

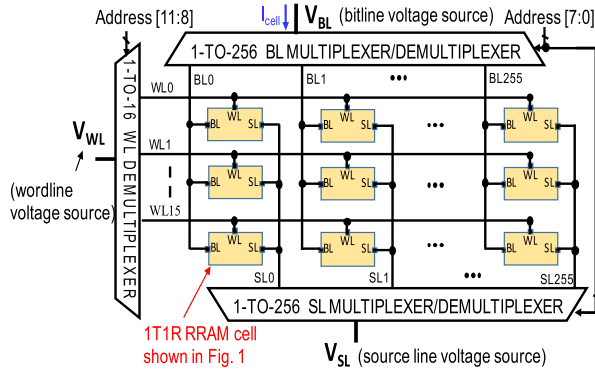


Fig. 3. 4-Kbit array architecture with single-cell access capability.

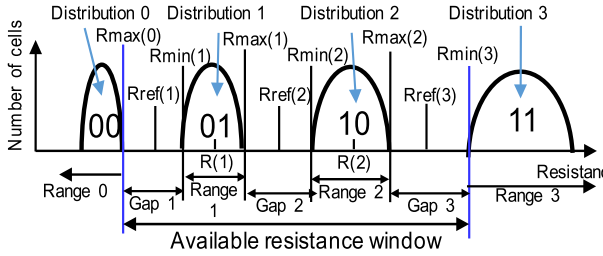


Fig. 4. Illustration of resistance range, resistance gap, resistance distribution, and resistance window for 2 bits-per-cell RRAM. Three reference resistors are needed. Reference resistors are routinely implemented for RRAM (e.g., [9]).

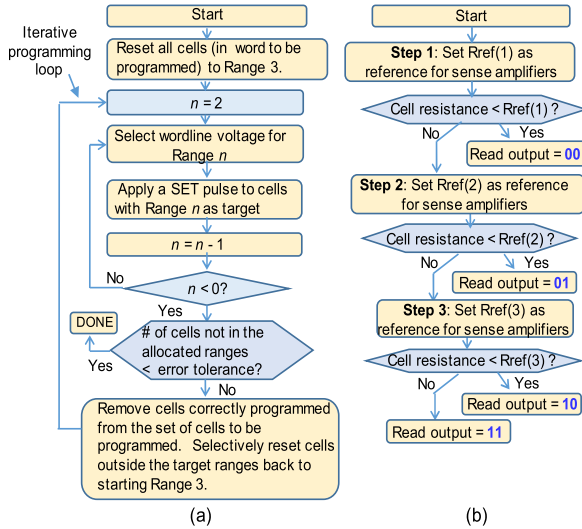


Fig. 5. Multiple bits-per-cell (a) write and (b) read operations, for 2 bits-per-cell RRAM. For our technology, the RESET operation applies a 100-ns pulse of -2.5 V across the BL and SL (i.e., $V_{BL} = 0$ V, $V_{WL} = 3$ V, $V_{SL} = 2.5$ V) to change the cell to the highest resistance range. A SET operation applies a 100-ns pulse of 2 V across the BL and SL (i.e., $V_{BL} = 2$ V, $V_{SL} = 0$ V) with V_{WL} chosen to change the cell resistance to the desired range. SET operation for each range is done sequentially, range 2 first (when $n = 2$), then range 1 (when $n = 1$) and finally range 0 (when $n = 0$).

A reference resistance value (from a reference resistor) is assigned in each resistance gap, $R_{ref}(1)$, $R_{ref}(2)$, and $R_{ref}(3)$ in Fig. 4. During read operation, the resistance of the cell being read is compared to the reference values to determine which resistance range it belongs to [Fig. 5(b)]. Resistance

TABLE I
COMPARISON TO PRIOR WORK. SINGLE CELL* = NOT CLEARLY STATED IN THE REFERENCE, THUS, WE ASSUME 1 IN THE TABLE

	Array Level?	bits per cell	# of cells measured	Constant ΔI for all gaps?	Allocate range efficiently
This work	Yes	3	7,746	Yes	Yes
[2]	No	3	standalone single cell *	No	No
[3]	No	3	standalone single cell *	No	No
[4]	No	6.5	standalone single cell *	No	No
[5]	Yes	2	40	No	No

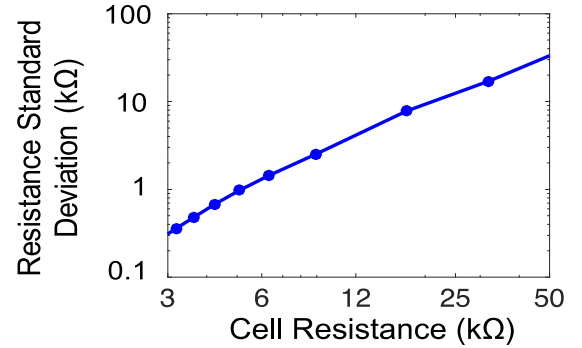


Fig. 6. Measured resistance standard deviation as a function of the resistance target for programming.

gaps should be allocated based on constant current margin ΔI (similar to I_{MARGIN} in [1], albeit for a different technology), to ensure similar sensing speed with any of the reference resistances. ΔI is the difference between the cell current I_{cell} and the current through the reference resistor I_{ref} , and its value is chosen based on the sensing speed requirement of a particular design. Constant current ΔI means (by Ohm's law):

$$\frac{V_{BL}}{R_{max}(0)} - \frac{V_{BL}}{R_{ref}(1)} = \frac{V_{BL}}{R_{ref}(1)} - \frac{V_{BL}}{R_{min}(1)} = \Delta I$$

[$R_{max}(0)$ is given, the lower bound of the available resistance window, and V_{BL} is the bitline voltage, Figs. 1 and 3]. Gap 1 = $R_{min}(1) - R_{max}(0)$. Resistance gaps are thus budgeted from the available resistance window such that

$$\begin{aligned} \frac{V_{BL}}{R_{max}(0)} - \frac{V_{BL}}{R_{min}(1)} &= \frac{V_{BL}}{R_{max}(1)} - \frac{V_{BL}}{R_{min}(2)} \\ &= \frac{V_{BL}}{R_{max}(2)} - \frac{V_{BL}}{R_{min}(3)} = 2\Delta I. \end{aligned}$$

R_{ref} 's are determined from the above-mentioned equations. We discuss allocation of resistance ranges (and derivation of R_{max} 's) in Sections II and III.

Prior work on RRAM with multiple bits per cell ([2]–[5], Table I, shown at the top of this page) can be categorized as follows: 1) [2]–[4] show the standalone single-cell level (i.e., not at the array level) demonstrations with 3–6.5 bits per cell and 2) [5] shows the write and read operations for

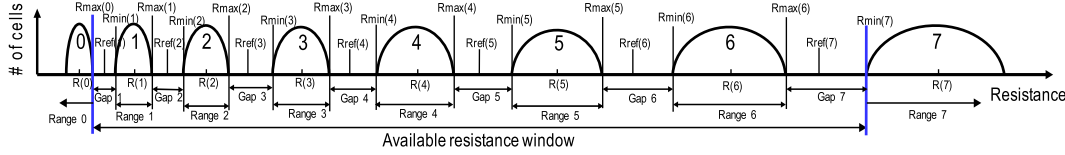


Fig. 7. Resistance range, resistance gap, resistance distribution, and resistance window for 3 bits-per-cell RRAM. Seven reference resistors (discussed in Fig. 4) are needed for read operation.

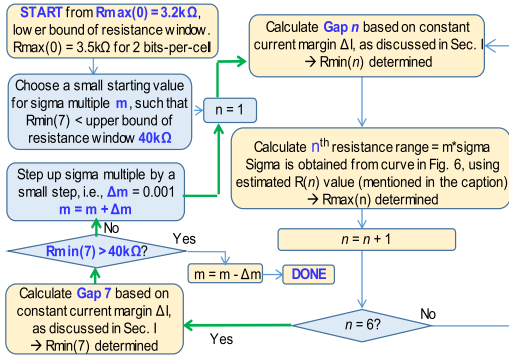


Fig. 8. Steps for calculations shown in Fig. 9. $R(n)$ can be estimated using $(1/2) \times m \times \sigma$, where σ is calculated from the curve in Fig. 6, using $Rmin(n)$.

(a)	Sigma multiple $m = 1.088$	V_{BL} (V)	Resistance range (n) (Ω)	$Rmin(n)$ (Ω)	$Rmax(n)$ (Ω)	# Programming (Pgrm.) Iterations (Fresh)
n = 0	N/A	N/A	N/A	N/A	3200	1
n = 1			532	3306	3838	7
n = 2			782	3991	4773	7
n = 3			1215	5012	6227	12
n = 4		0.20	2034	6640	8675	13
n = 5			3793	9498	13292	9
n = 6			13192	15329	28521	8
n = 7			N/A	39902	N/A	N/A
				Mean:		9

(b)	Sigma multiple $m = 1.205$	V_{BL} (V)	Resistance range (n) (Ω)	$Rmin(n)$ (Ω)	$Rmax(n)$ (Ω)	# Pgrm. Iterations (Fresh)	# Pgrm. Iterations (4k cycles)
n = 0	N/A	N/A	N/A	N/A	3200	1	1
n = 1		0.20	597	3306	3903	6	10
n = 2		0.22	903	4045	4949	6	11
n = 3		0.23	1454	5174	6627	8	9
n = 4		0.24	2538	7020	9559	8	15
n = 5		0.25	4968	10341	15310	9	15
n = 6		0.28	16164	17154	33318	8	12
n = 7		0.40	N/A	39978	N/A	N/A	N/A
				Mean:		7	11

(c)	Sigma multiple $m = 4.18$	V_{BL} (V)	Resistance range (n) (Ω)	$Rmin(n)$ (Ω)	$Rmax(n)$ (Ω)	# Pgrm. Iterations (Fresh)	# Pgrm. Iterations (4k cycles)
n = 0	N/A	N/A	N/A	N/A	3500	1	1
n = 1		0.20	3790	3763	7553	3	5
n = 2		0.25	19974	8583	28556	4	3
n = 3		0.40	N/A	39970	N/A	N/A	N/A
				Mean:		3	3

Fig. 9. 3 bits-per-cell allocation based on (a) SBA alone (with $V_{BL} = 0.2$ V) and (b) SBA + BVA combined. Current margin $\Delta I = 1 \mu A$. Resistance range 6 is slightly modified for lognormal distribution since its median is $> 12.9 k\Omega$ [7]. (c) 2 bits-per-cell allocation with SBA + BVA with current margin $\Delta I = 3 \mu A$ for higher sensing speed and $Rmax(0) = 3.5 k\Omega$ to reduce sensing current. N/A: Not applicable.

40 cells (2 bits per cell) in a 1-Kbit array. The system-level implementation of multiple bits-per-cell RRAM requires a deep understanding of the interactions between a wide variety

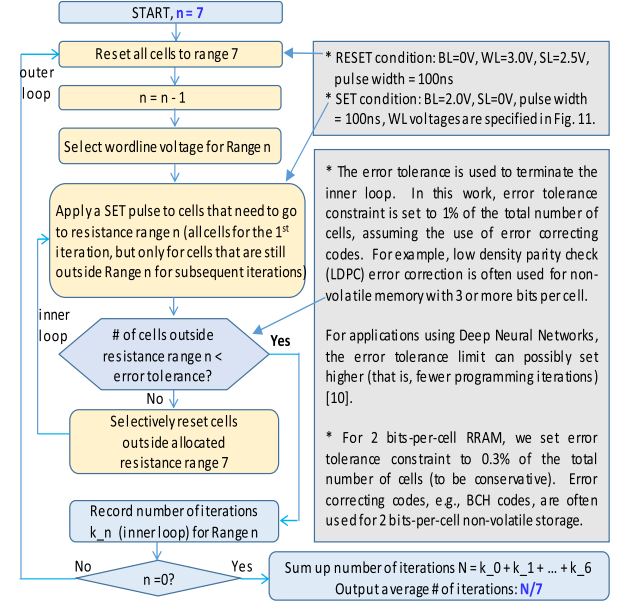


Fig. 10. Procedure to obtain measured data in Figs. 11 and 12. Inner loop is used to count the number of iterations for each range, and outer loop is used to step through all seven resistance ranges to demonstrate 3 bits-per-cell capability.

of parameters, e.g., resistance gaps, resistance ranges, resistance distributions, programming iterations, bitline voltages, and wordline voltages. In this paper, we present an array-level demonstration of RRAM with 3 bits per cell (2 bits per cell as well) over multiple arrays. Such demonstration is enabled by the coordinated use of techniques in Sections II and III (in conjunction with constant sensing margin to derive resistance gaps, discussed earlier in this section).

II. RESISTANCE RANGE ALLOCATION BASED ON RRAM VARIATIONS

We use the standard deviation(s) of resistance values (referred to as sigma(s) in this paper) of RRAM cells to allocate resistance ranges. Hence, we call this technique sigma-based allocation (SBA). To illustrate SBA, consider the 2 bits-per-cell RRAM case for which we need to allocate resistance ranges 1 and 2. Suppose that we want to program the contents of a group G of cells (in a memory word) to, say, bit combination “10” corresponding to resistance range 2 (Fig. 4). Following the programming procedure in Fig. 5(a) (similar to [6]), all cells of the word are first RESET to range 3. Next, a SET pulse is applied to cells in G , trying to set their resistances to values within range 2. Cells in G that still have their resistances outside range 2 are RESET back to range 3 before another SET pulse is applied. Such sequence

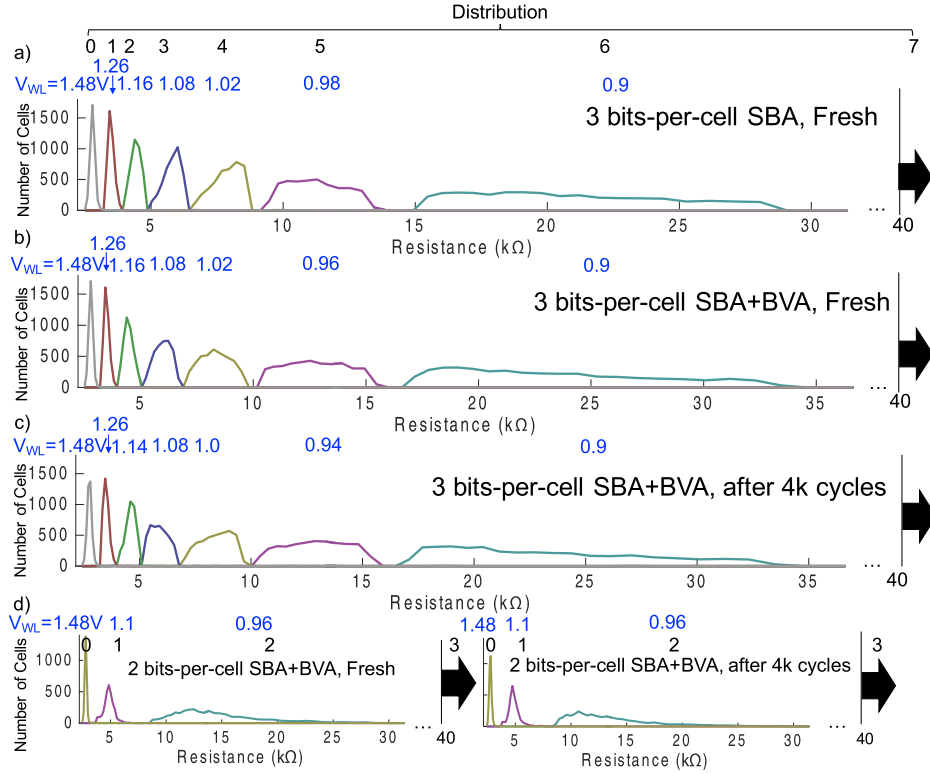


Fig. 11. Resistance distribution measured data, obtained from the procedure specified in detail in Fig. 10. Range and gap allocation for (a), 3 bits-per-cell SBA, Fresh, is from Fig. 9(a), for (b)/(c), 3 bits-per-cell SBA+BVA, Fresh/3 bits-per-cell SBA+BVA, after 4k cycles, is from Fig. 9(b), and for (d), 2 bits-per-cell SBA+BVA, is from Fig. 9(c). The plots in this figure show that we could achieve 3 bits-per-cell capability with eight distinct distributions [or four distributions for 2 bits-per-cell in (d)], even after 4K cycles. The number of iterations needed to achieve these plots is specified in Fig. 12. Due to RRAM select transistor threshold variations, compliance current of 5.4% of cells cannot be controlled accurately when V_{WL} is low (can be controlled accurately by current source, but not available on our array). We filtered these cells, and thus, we use 7746 cells (over two arrays) for this paper. One cycle = program from the highest resistance range to lowest and back.

of RESET followed by SET is a programming iteration. For a given resistance window (~ 37 k Ω for our RRAM technology, from 3.2 to 40 k Ω), we need an effective strategy to allocate resistance ranges appropriately to minimize programming iterations (ideally 1). For example, suppose that resistance gaps 1–3 (Fig. 4) all together consume 13 k Ω (allocated as explained in Section I). How do we allocate the remaining 24 k Ω to resistance ranges 1 and 2?

For RRAM, the variance in cell resistance values is generally higher for higher cell resistance values ([7], [8], also confirmed by measured data in Fig. 6 collected using our arrays). SBA uses this fact to allocate a wider range 2 versus range 1. Suppose that sigmas of resistance values in ranges 1 and 2 are 0.65 k Ω (mean: ~ 4 k Ω) and 3.33 k Ω (mean: ~ 12 k Ω), respectively (Fig. 6). If SBA allocates 4 k Ω for range 1, it covers ± 3 sigma around the corresponding mean ($(4 \text{ k}\Omega / 0.65 \text{ k}\Omega) \approx 6\text{sigmas}$, *i.e.*, $\pm 3\text{sigmas}$)—there is about 99.7% chance a cell can be programmed to range 1 in a single-programming iteration (assuming Gaussian distribution for resistance values in range 1; as [7] explains in detail, the distribution is approximately Gaussian when mean value < 12.9 k Ω ; otherwise, it is approximately lognormal. The relationship shown in Fig. 6 of this paper is also captured in Fig. 7 and [7, Table III]. SBA then allocates 20 k Ω (24–4 k Ω) to range 2, which also covers ± 3 sigma around the corresponding mean ($(20 \text{ k}\Omega / 3.33 \text{ k}\Omega) \approx 6\text{sigmas}$). Contrast

SBA versus a naive approach that allocates 12 k Ω each to ranges 1 and 2. That would correspond to ± 9 sigma for range 1 (12 k $\Omega / 0.65 \text{ k}\Omega$), but only ± 1.8 sigma for range 2, substantially increasing the number of programming iterations for range 2. Fig. 7 illustrates similar quantities as in Fig. 4, but for 3 bits-per-cell case. Fig. 8 shows the steps to allocate resistance ranges based on SBA [corresponding results for our 3 bits-per-cell RRAM demonstration in Fig. 9(a) by using sigmas (obtained from Fig. 6) and a factor m (sigma multiple)], which is chosen to be as large as possible while ensuring that the total allocation of resistance ranges and gaps does not exceed the resistance window. SBA is different from techniques used for FLASH (NAND or NOR), where ranges (in terms of threshold voltage) are about the same.

III. BITLINE VOLTAGE ALLOCATION

The bitline voltage allocation (BVA) technique in this section, when applied in conjunction with SBA (Section II), helps reduce the average number of programming iterations further (vs. SBA alone). The key idea is to adjust the bitline voltage V_{BL} (Figs. 1 and 3) depending on the reference level R_{ref} [steps 1, 2, or 3 in Fig. 5(b)]. The overall effect is resistance ranges increase (fewer programming iterations) while resistance gaps decrease (without impacting current margin ΔI). Consider the equation $(V_{BL}/R_{max}(0)) - (V_{BL}/R_{min}(1)) = 2\Delta I$ (Section I). For $V_{BL} = 0.2$ V,

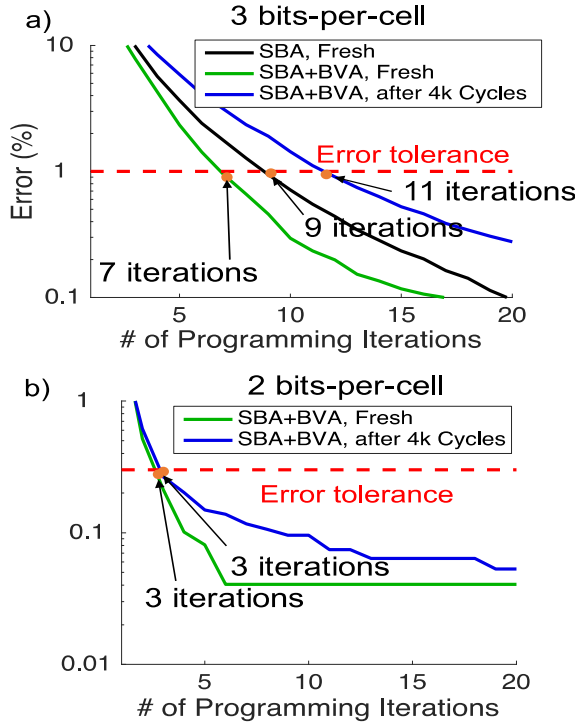


Fig. 12. Measured average number of programming iterations for (a) 3 bits-per-cell case with error tolerance set at 1% and (b) 2 bits-per-cell case with error tolerance set at 0.3%. Three curves in (a) correspond to Fig. 11(a)–(c), and also Fig. 9(a) and (b). With both techniques SBA and BVA applied, the number of programming iterations reduces from 9 to 7, compared to applying only SBA. After 4K cycles, the number of programming iterations is still only 11, compared to more than 20 (at fresh) for the “naïve allocation” mentioned at the end of Section III. The curves in (b) correspond to Figs. 11(d) and 9(c).

$\Delta I = 2 \text{ } \Omega\text{A}$ and $R_{\max}(0) = 3500 \text{ } \Omega$, $R_{\min}(1) = 3763 \text{ } \Omega$, and gap 1 = $263 \text{ } \Omega$. For the same current margin (ΔI), increasing V_{BL} to 0.4 V makes $R_{\min}(1) = 3627 \text{ } \Omega$ and gap 1 = $127 \text{ } \Omega$ (at the cost of doubling $(V_{BL}/R_{\text{ref}}(1)) = I_{\text{ref}}$ from 55 to $110 \text{ } \mu\text{A}$).

The issue of increased current is mitigated by choosing the amount of V_{BL} increase (during read operation) depending on the value of the reference resistance used for sensing. For example, in Fig. 4, suppose that $R_{\text{ref}}(1)$, $R_{\text{ref}}(2)$, and $R_{\text{ref}}(3)$ are 3600, 7200, and 32400 Ω , respectively, and that V_{BL} can be regulated in the range from $V_{BL\min} = 0.2 \text{ V}$ to $V_{BL\max} = 0.4 \text{ V}$. During step 1 in Fig. 5(b), when $R_{\text{ref}}(1)$ is used, the reference current is $(V_{BL} = 0.2\text{V}/R_{\text{ref}}(1) = 3600\Omega) = 55.5 \text{ } \mu\text{A}$; therefore, we do not increase V_{BL} at this step. However, during step 3 in Fig. 5(b), when $R_{\text{ref}}(3)$ is used, the current is $(V_{BL} = 0.2\text{V}/R_{\text{ref}}(3) = 32,400\Omega) = 6.17 \text{ } \mu\text{A}$; therefore, we can use $V_{BL} = 0.4 \text{ V}$, since the current increases by only $6.17 \text{ } \mu\text{A}$. During step 2 of Fig. 5(b), when $R_{\text{ref}}(2)$ is used, we pick V_{BL} in proportion to R_{refs} , i.e.,

$$V_{BL} = V_{BL\min} + \left(\frac{R_{\text{ref}}(2)}{R_{\text{ref}}(3)} \right) (V_{BL\max} - V_{BL\min}) = 0.24 \text{ V}.$$

Fig. 9(b) shows the combined benefits of SBA + BVA versus SBA alone [Fig. 9(a)] for our 3 bits-per-cell RRAM demonstration: we improve the sigma multiple m from

1.199 to 1.356, resulting in fewer programming iterations on average (e.g., from 9 to 7, detailed measured data in Fig. 11). In contrast, if resistance ranges are naively allocated equally, more than 7% of cells continue to be erroneous even after 20 programming iterations. Fig. 10 shows our extensive testing procedure (7746 cells, all possible bit combinations in each cell, at fresh and after 4K cycles of operation) to obtain our measured data in Figs. 11 and 12, demonstrating correct operation of 3 bits-per-cell (2 bits-per-cell as well) RRAM at the array level.

IV. CONCLUSION

We have demonstrated, for the first time, RRAM with 3 bits per cell at the array level (and 2 bits per cell as well), together with read and write techniques that can have circuit realizations in systems today. Such first demonstration is made possible through the combined use of SBA and BVA techniques introduced in this paper. Our techniques can be used for general storage applications [13] as well as for on-chip memory in next-generation computing systems [11]. Future directions include the following.

Array-level

- 1) demonstration of RRAM with more than 3 bits per cell (SBA, BVA, and resistance gap allocation can be used for more than 3 bits per cell).
- 2) Understanding interdependencies between materials for RRAM fabrication [13], programming iterations (hence, write energy and time), coding techniques for detecting/correcting residual errors, and application-level needs (e.g., storage systems where codes with longer latencies may be tolerated versus computing systems such as [11] where shorter latencies may be required).
- 3) Exploration of tradeoffs between the number of bits per RRAM cell versus allocation of resistance gaps and resistance ranges in the presence of variations versus application-level effects of errors, especially for machine learning (traditional algorithms [11], deep learning [10], or other brain-inspired computing models [12]) or graph analytics that operate on massive amounts of data.

ACKNOWLEDGMENT

The authors would like to thank X. Zheng and Prof. S. Wong from Stanford University, Stanford, CA, USA, and Prof. C. Zambelli from the University of Ferrara, Ferrara, Italy, for fruitful discussions.

REFERENCES

- [1] H. Pozidis *et al.*, “Enabling technologies for multilevel phase-change memory,” in *Proc. Eur. Phase Change Ovonic Symp.*, 2011, pp. 1–8.
- [2] W.-C. Chien *et al.*, “Multi-level 40nm WO_x resistive memory with excellent reliability,” in *IEDM Tech. Dig.*, Dec. 2011, pp. 31.5.1–31.5.4, doi: [10.1109/IEDM.2011.6131651](https://doi.org/10.1109/IEDM.2011.6131651).
- [3] A. Prakash, J. Park, J. Song, J. Woo, E.-J. Cha, and H. Hwang, “Demonstration of low power 3-bit multilevel cell characteristics in a TaO_x-based RRAM by stack engineering,” *IEEE Electron Device Lett.*, vol. 36, no. 1, pp. 32–34, Jan. 2015, doi: [10.1109/LED.2014.2375200](https://doi.org/10.1109/LED.2014.2375200).
- [4] S. Stathopoulos, A. Khayat, M. Trapatseli, S. Cortese, A. Serb, and I. V. T. Prodromakis, “Multibit memory operation of metal-oxide bilayer memristors,” *Sci. Rep.*, vol. 7, 2017, Art. no. 17532, doi: [10.1038/s41598-017-17785-1](https://doi.org/10.1038/s41598-017-17785-1).

- [5] S.-S. Sheu *et al.*, "A 5ns fast write multi-level non-volatile 1 K bits RRAM memory with advance write scheme," in *Proc. Symp. VLSI Circuits*, Jun. 2009, pp. 82–83.
- [6] M.-H. Lee *et al.*, "Resistance control of transition metal oxide resistive memory (TMO ReRAM)," in *Proc. Int. Conf. Solid-State Integr. Circuit Technol. (ICSICT)*, Oct. 2016, pp. 182–185, doi: [10.1109/ICSICT.2016.7998873](https://doi.org/10.1109/ICSICT.2016.7998873).
- [7] A. Grossi *et al.*, "Fundamental variability limits of filament-based RRAM," in *IEDM Tech. Dig.*, Dec. 2016, pp. 4.7.1–4.7.4, doi: [10.1109/IEDM.2016.7838348](https://doi.org/10.1109/IEDM.2016.7838348).
- [8] S. Ambrogio, S. Balatti, A. Cubeta, A. Calderoni, N. Ramaswamy, and D. Ielmini, "Statistical fluctuations in HfO_x resistive-switching memory: Part I—Set/reset variability," *IEEE Trans. Electron Devices*, vol. 61, no. 8, pp. 2912–2919, Aug. 2014, doi: [10.1109/TED.2014.2330200](https://doi.org/10.1109/TED.2014.2330200).
- [9] M.-F. Chang *et al.*, "19.4 embedded 1Mb ReRAM in 28nm CMOS with 0.27-to-1V read using swing-sample-and-couple sense amplifier and self-boost-write-termination scheme," in *IEEE ISSCC Dig. Tech. Papers*, Feb. 2014, pp. 332–333, doi: [10.1109/ISSCC.2014.6757457](https://doi.org/10.1109/ISSCC.2014.6757457).
- [10] M. Qin, C. Sun, and D. Vucinic. (2017). "Robustness of neural networks against storage media errors." [Online]. Available: <https://arxiv.org/abs/1709.06173>
- [11] M. M. S. Aly *et al.*, "Energy-efficient abundant-data computing: The N3XT 1,000x," *Computer*, vol. 48, no. 12, pp. 24–33, Dec. 2015, doi: [10.1109/MC.2015.376](https://doi.org/10.1109/MC.2015.376).
- [12] T. F. Wu *et al.*, "Brain-inspired computing exploiting carbon nanotube FETs and resistive RAM: Hyperdimensional computing case study," in *IEEE ISSCC Dig. Tech. Papers*, Feb. 2018, pp. 492–494, doi: [10.1109/ISSCC.2018.8310399](https://doi.org/10.1109/ISSCC.2018.8310399).
- [13] H.-S. P. Wong and S. Salahuddin, "Memory leads the way to better computing," *Nature Nanotechnol.*, vol. 10, pp. 191–194, Mar. 2015, doi: [10.1038/nnano.2015.29](https://doi.org/10.1038/nnano.2015.29).

Authors' photographs and biographies not available at the time of publication.